

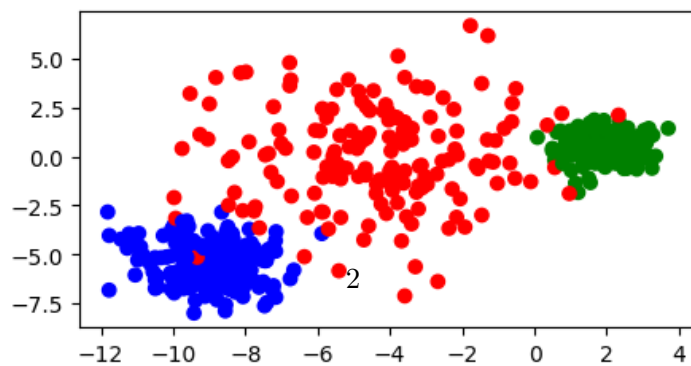
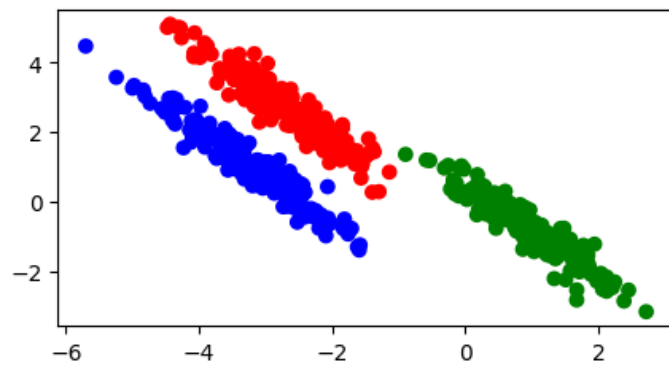
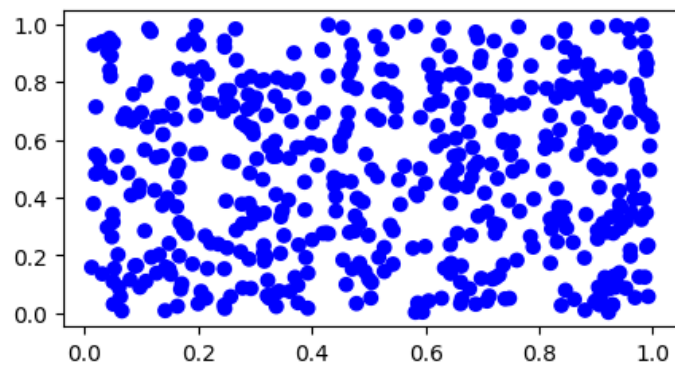
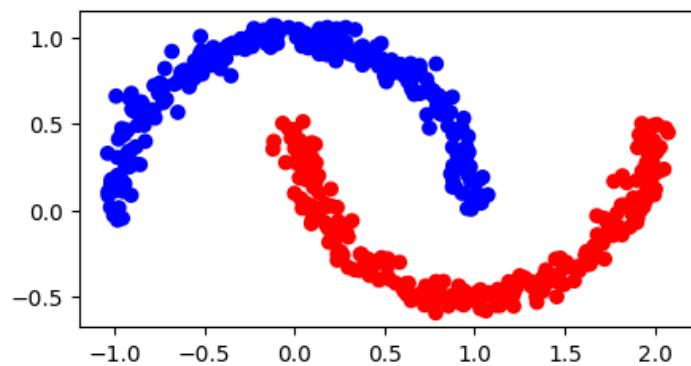
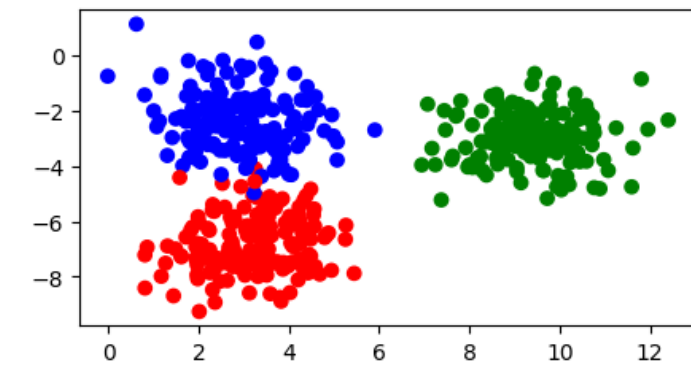
Clustering

November 3, 2025

```
[1]: import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn import cluster
from sklearn import datasets
from sklearn.preprocessing import MinMaxScaler, StandardScaler
from sklearn.cluster import KMeans
from sklearn.cluster import DBSCAN

[2]: nSamples = 500
seed = 30
noisyCircles = datasets.make_circles(n_samples=nSamples, factor=.5, noise=.05,
    random_state=seed)
noisyMoons = datasets.make_moons(n_samples=nSamples, noise=.05,
    random_state=seed)
blobs = datasets.make_blobs(n_samples=nSamples, random_state=seed)
rng = np.random.RandomState(seed)
noStructure = rng.rand(nSamples, 2), np.zeros(nSamples, np.int8)
randomState = 170
x , y = datasets.make_blobs(n_samples=nSamples, random_state=randomState)
transformation = [[0.6,-0.6],[-0.4,0.8]]
Xaniso = np.dot(x, transformation)
aniso = (Xaniso, y)
varied = datasets.make_blobs(n_samples = nSamples, cluster_std = [1.1, 2.5, 0.
    7] , random_state=randomState)

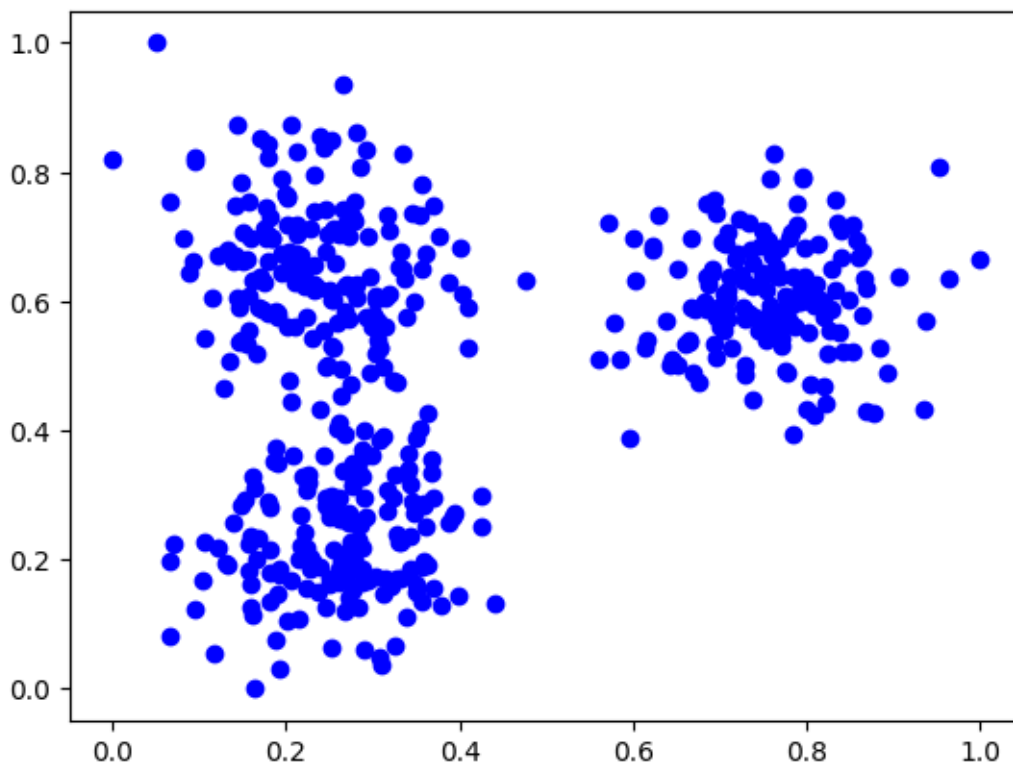
[3]: fig , ax = plt.subplots(5 , 1 , figsize = (5,15))
color = np.array(["Blue","Red","Green"])
for (i,datasets) in enumerate([blobs , noisyMoons , noStructure , aniso ,
    varied]):
    ax[i].scatter(datasets[0][:,0], datasets[0][:,1], c=color[datasets[1]])
```



0.1 Preprocessing

```
[4]: dataset = blobs
x , y = dataset
x = MinMaxScaler().fit_transform(x)
plt.scatter(x[:, 0], x[:, 1], alpha=1.0, color='b')
```

```
[4]: <matplotlib.collections.PathCollection at 0x1e7f1e3fc50>
```

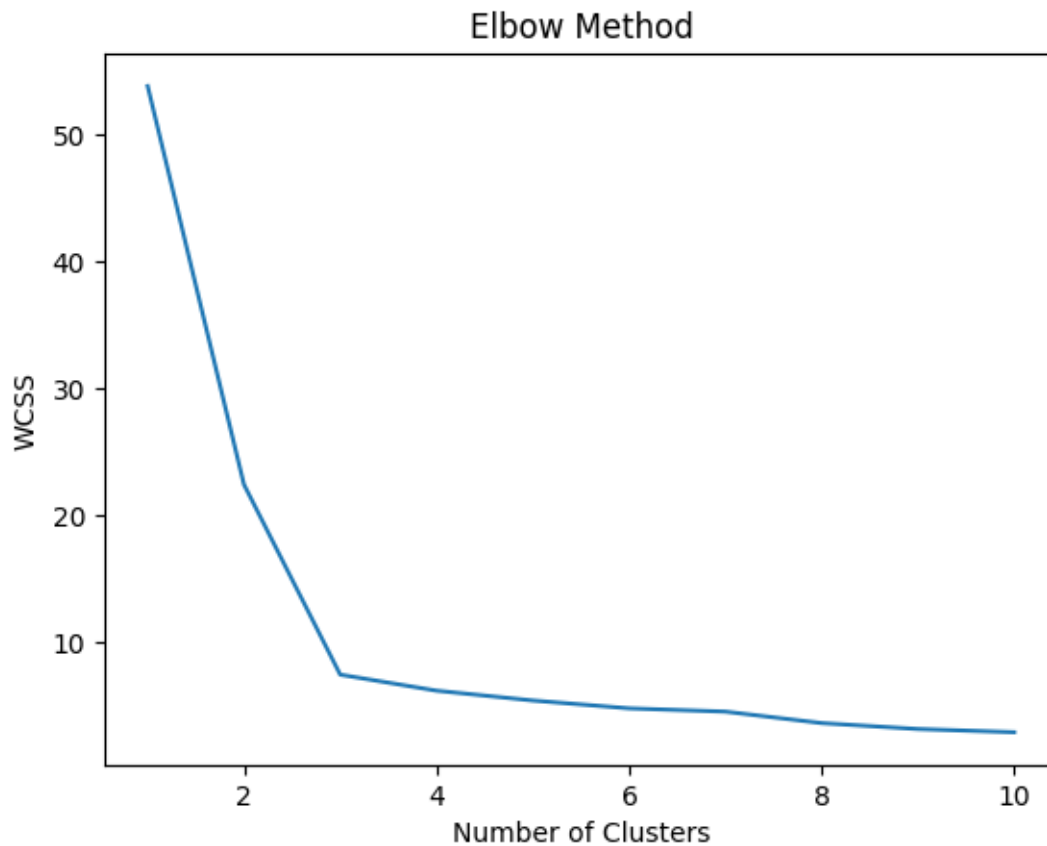


0.2 Clustering

0.2.1 Within-Cluster Sum of Squares (WCSS) metric

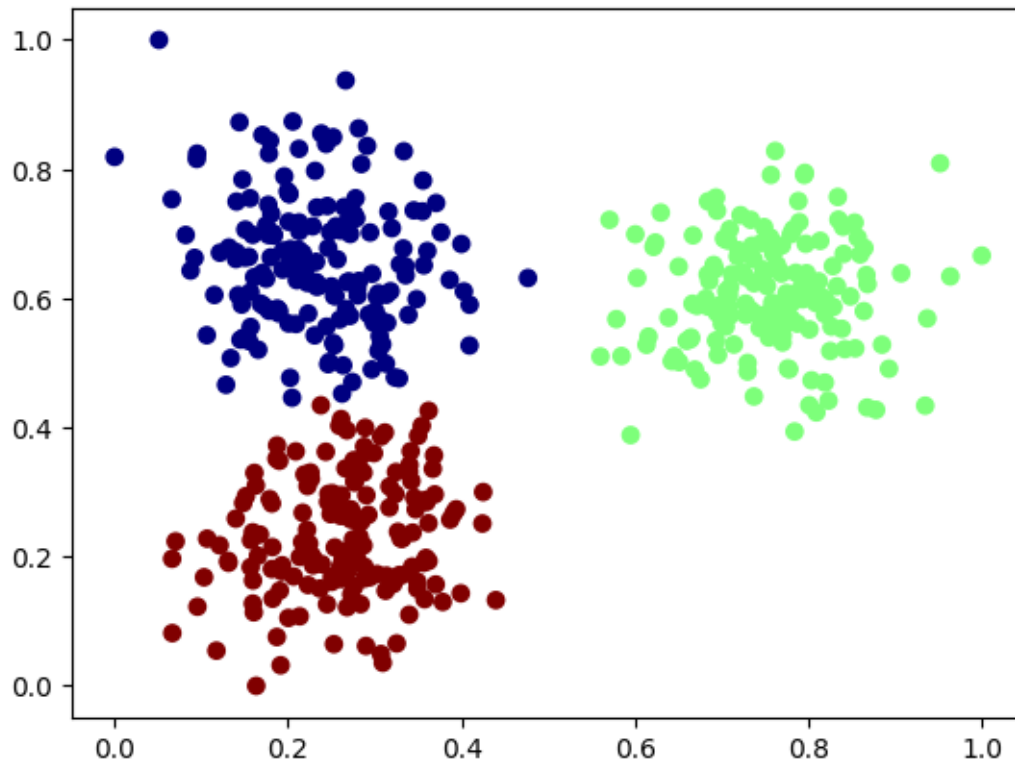
```
[5]: wcss = []
model = []
for i in range(1, 11):
    model.append(KMeans(n_clusters=i).fit(x))
    wcss.append(model[i-1].inertia_)
```

```
[6]: plt.plot(range(1, 11), wcss)
plt.title('Elbow Method')
plt.xlabel('Number of Clusters')
plt.ylabel('WCSS')
plt.show()
```



```
[7]: yp = model[2].fit_predict(x)
cmap = plt.cm.jet
fig , ax = plt.subplots()
ax.scatter(x[:, 0], x[:, 1], c=yp, cmap=cmap)
```

```
[7]: <matplotlib.collections.PathCollection at 0x1e7efab34d0>
```

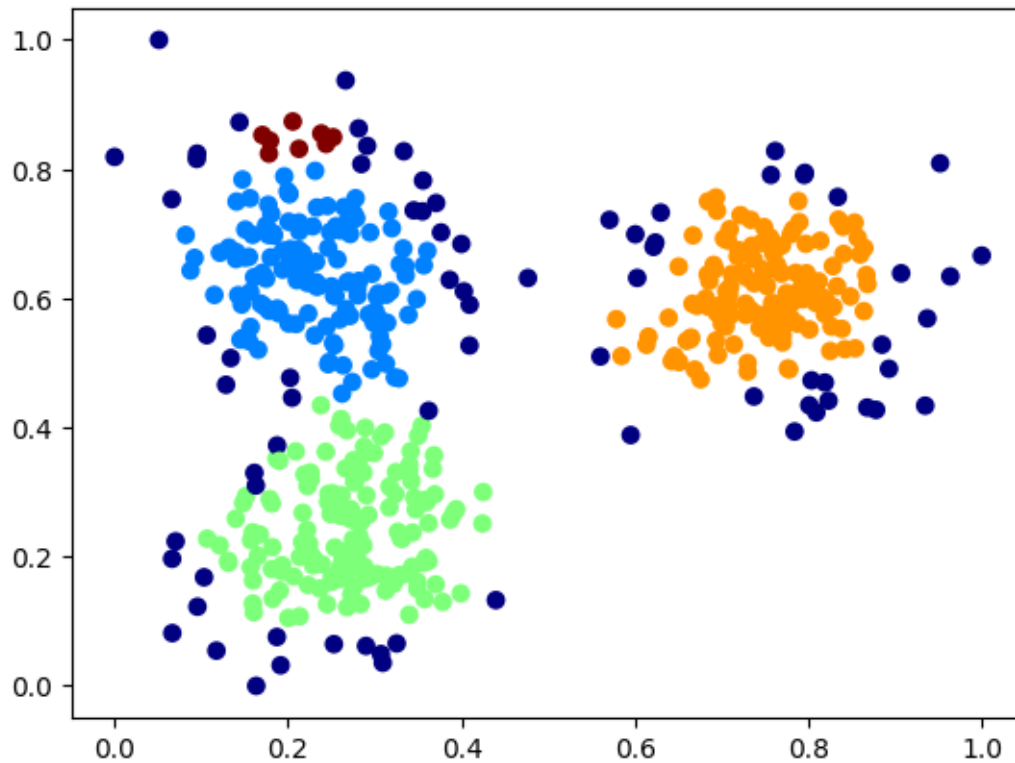


0.3 DBSCAN

0.3.1 Density-Based Spatial Clustering Applications With Noise

```
[8]: model = DBSCAN(eps=0.05, min_samples=10)
      yp = model.fit_predict(x)
      fig, ax = plt.subplots()
      ax.scatter(x[:, 0], x[:, 1], c=yp + 1, cmap=cmap)
```

```
[8]: <matplotlib.collections.PathCollection at 0x1e7efb58f50>
```



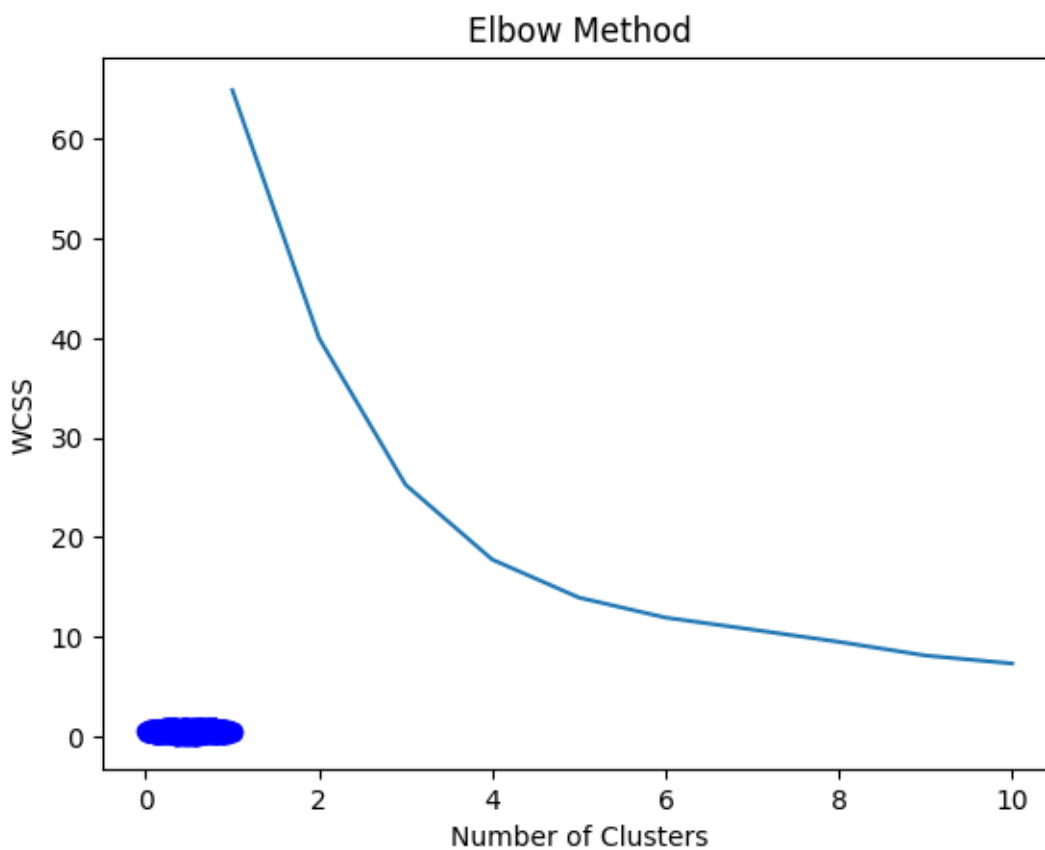
1 The same way for other datasets

```
[19]: dataset = noisyCircles
x , y = dataset
x = MinMaxScaler().fit_transform(x)
plt.scatter(x[:, 0], x[:, 1], alpha=1.0, color='b')
# Clustering
# Within-Cluster Sum of Squares (WCSS) metric
wcss = []
model = []
for i in range(1, 11):
    model.append(KMeans(n_clusters=i).fit(x))
    wcss.append(model[i-1].inertia_)
wcss = []
model = []
for i in range(1, 11):
    model.append(KMeans(n_clusters=i).fit(x))
    wcss.append(model[i-1].inertia_)
plt.plot(range(1, 11), wcss)
plt.title('Elbow Method')
plt.xlabel('Number of Clusters')
```

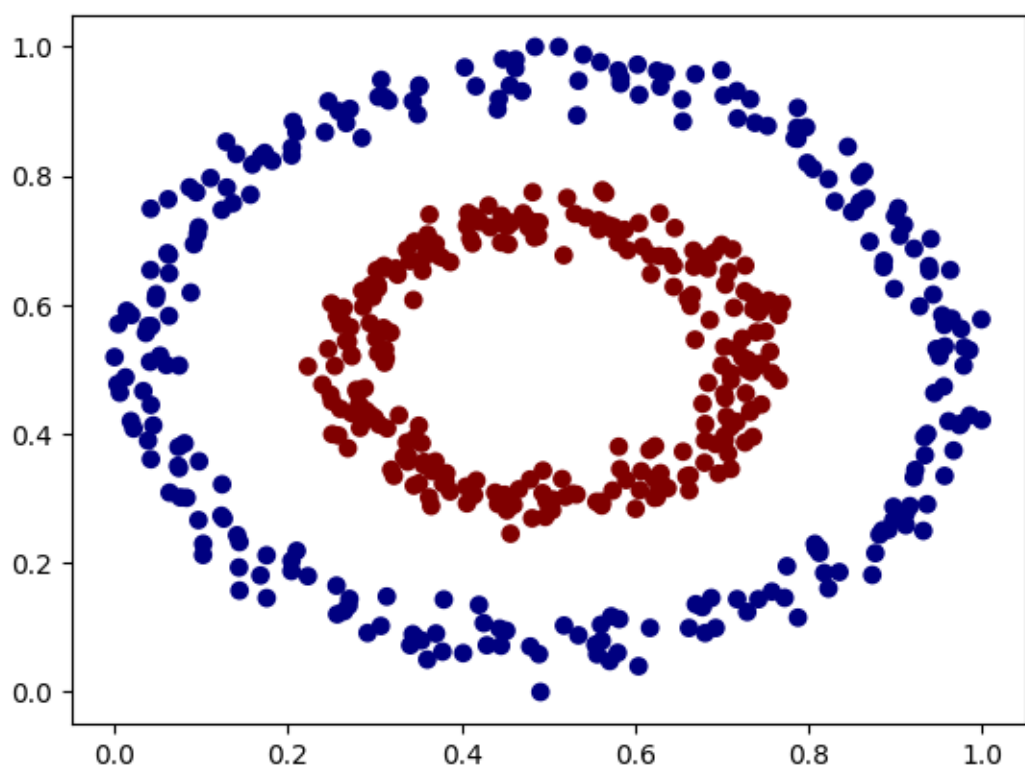
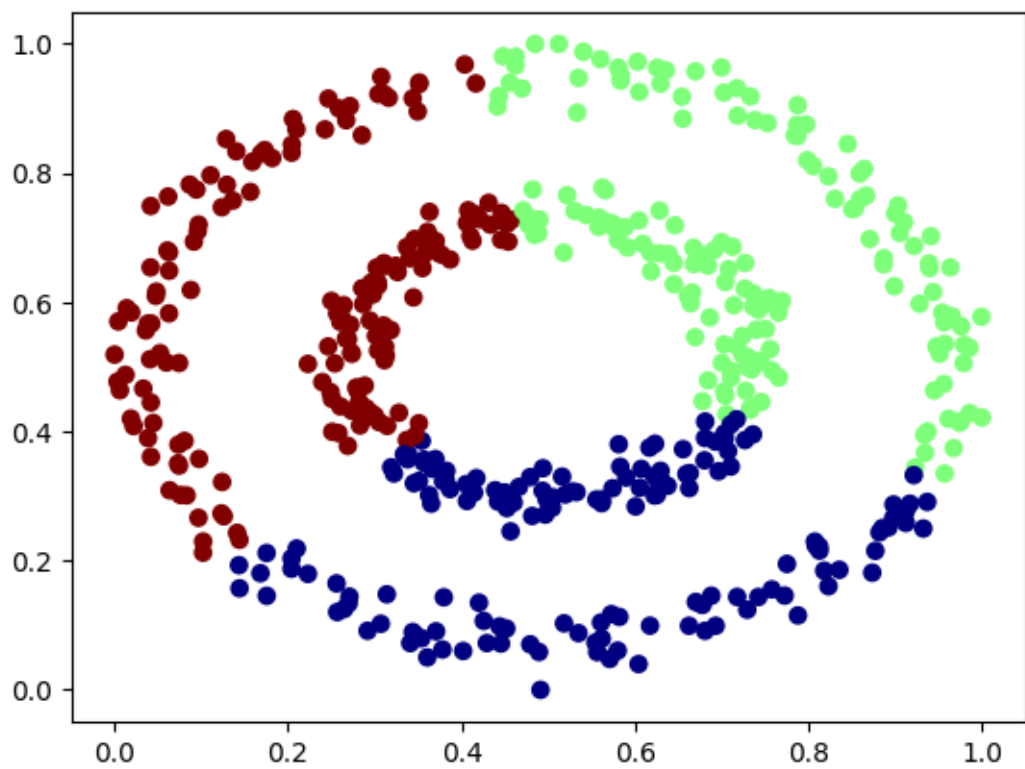
```

plt.ylabel('WCSS')
plt.show()
yp = model[2].fit_predict(x)
cmap = plt.cm.jet
fig , ax = plt.subplots()
ax.scatter(x[:, 0], x[:, 1], c=yp, cmap=cmap)
# DBSCAN
# Density-Based Spatial Clustering Applications With Noise
model = DBSCAN(eps=0.1, min_samples=10)
yp = model.fit_predict(x)
fig , ax = plt.subplots()
ax.scatter(x[:, 0], x[:, 1], c=yp + 1, cmap=cmap)

```

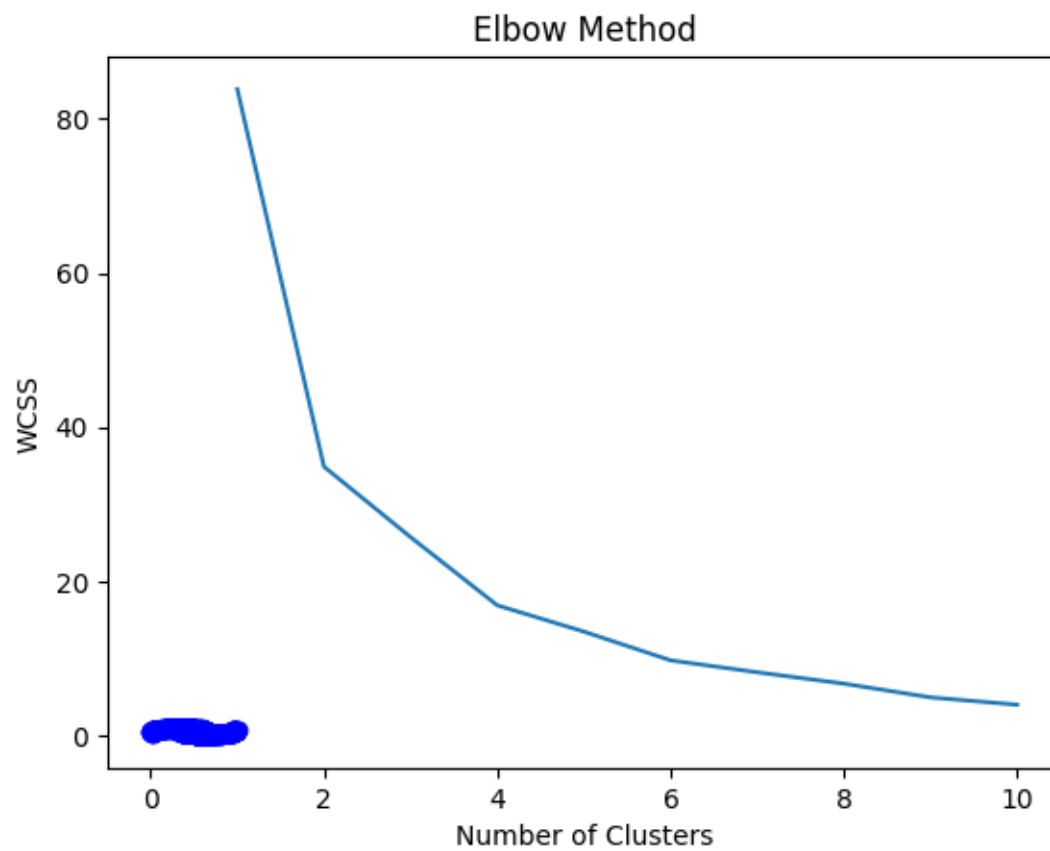


[19]: <matplotlib.collections.PathCollection at 0x1e7ef961d10>

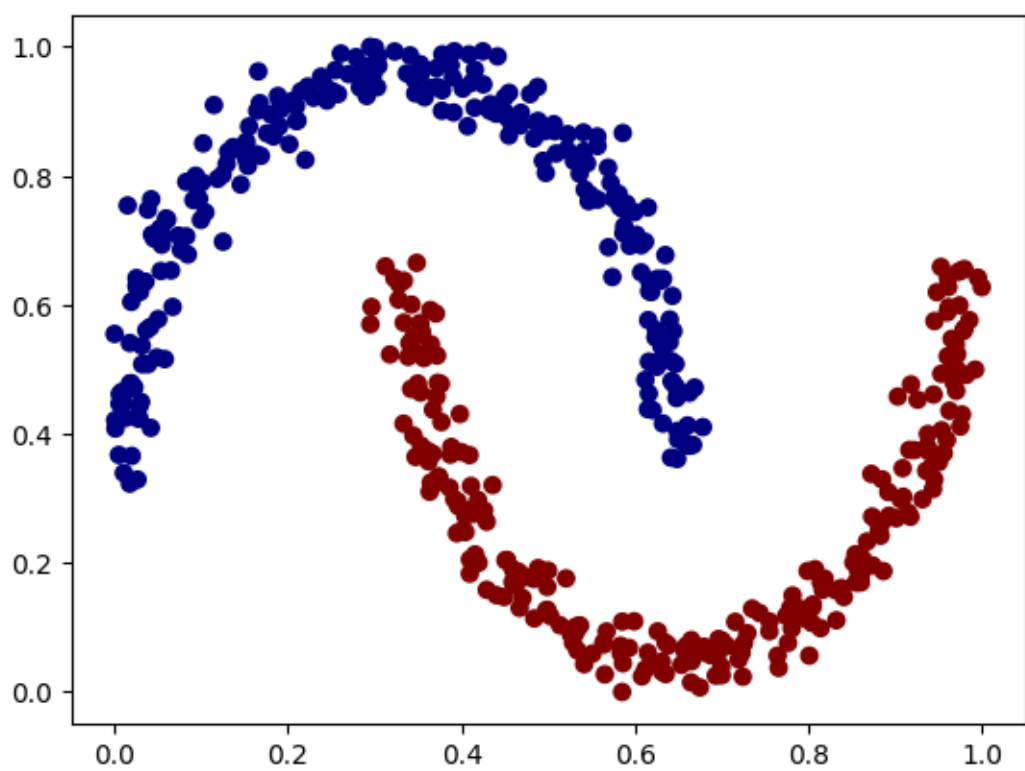
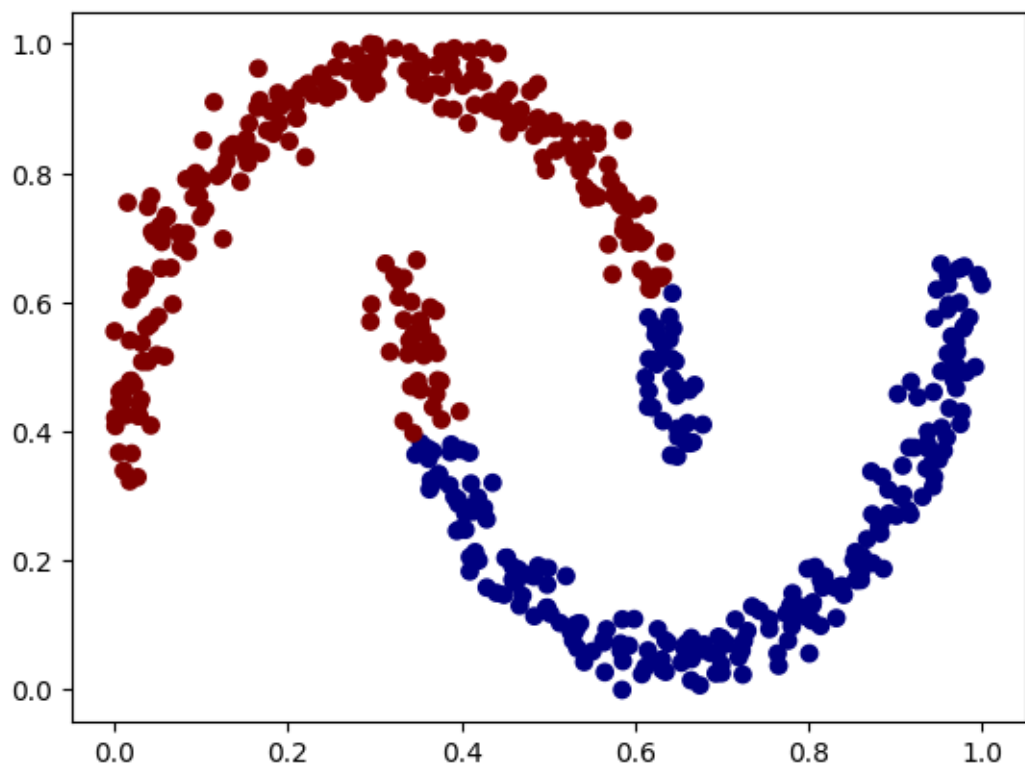


1.1 Noisy Moon

```
[10]: dataset = noisyMoons
x , y = dataset
x = MinMaxScaler().fit_transform(x)
plt.scatter(x[:, 0], x[:, 1], alpha=1.0, color='b')
# Clustering
# Within-Cluster Sum of Squares (WCSS) metric
wcss = []
model = []
for i in range(1, 11):
    model.append(KMeans(n_clusters=i).fit(x))
    wcss.append(model[i-1].inertia_)
plt.plot(range(1, 11), wcss)
plt.title('Elbow Method')
plt.xlabel('Number of Clusters')
plt.ylabel('WCSS')
plt.show()
yp = model[1].fit_predict(x)
cmap = plt.cm.jet
fig , ax = plt.subplots()
ax.scatter(x[:, 0], x[:, 1], c=yp, cmap=cmap)
# DBSCAN
# Density-Based Spatial Clustering Applications With Noise
model = DBSCAN(eps=0.065, min_samples=10)
yp = model.fit_predict(x)
fig , ax = plt.subplots()
ax.scatter(x[:, 0], x[:, 1], c=yp + 1, cmap=cmap)
```



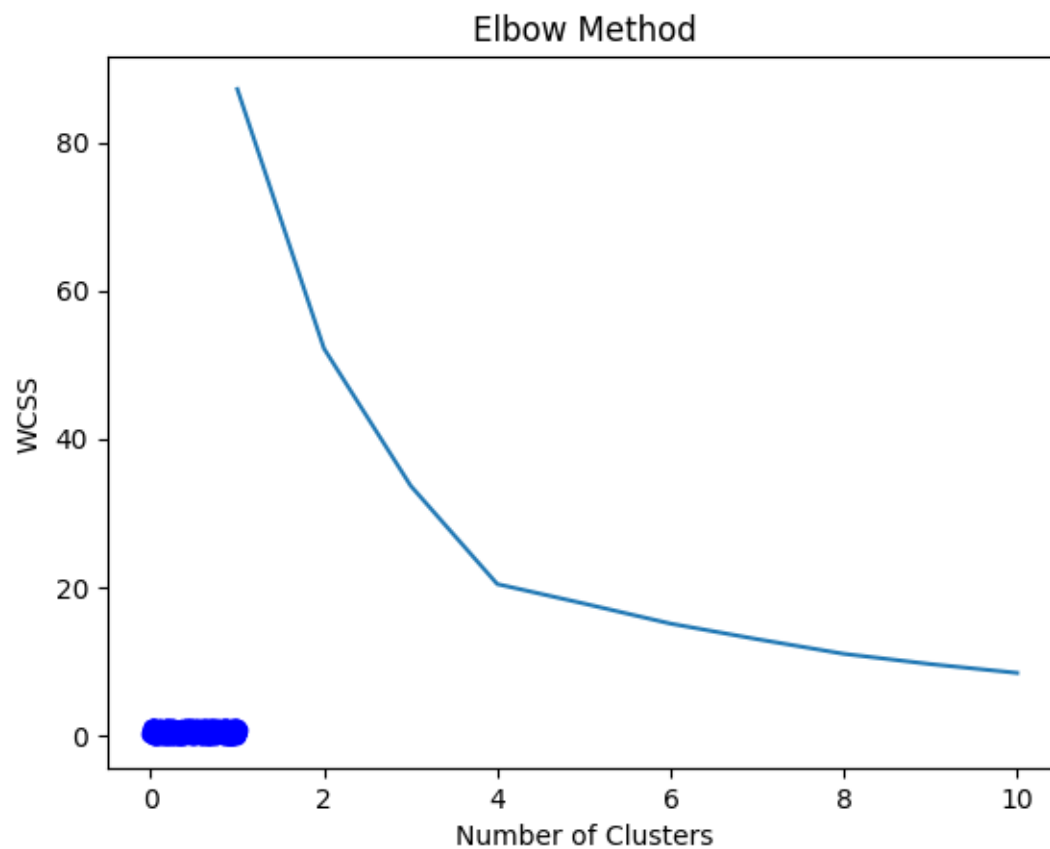
[10]: <matplotlib.collections.PathCollection at 0x1e7f63fbc50>



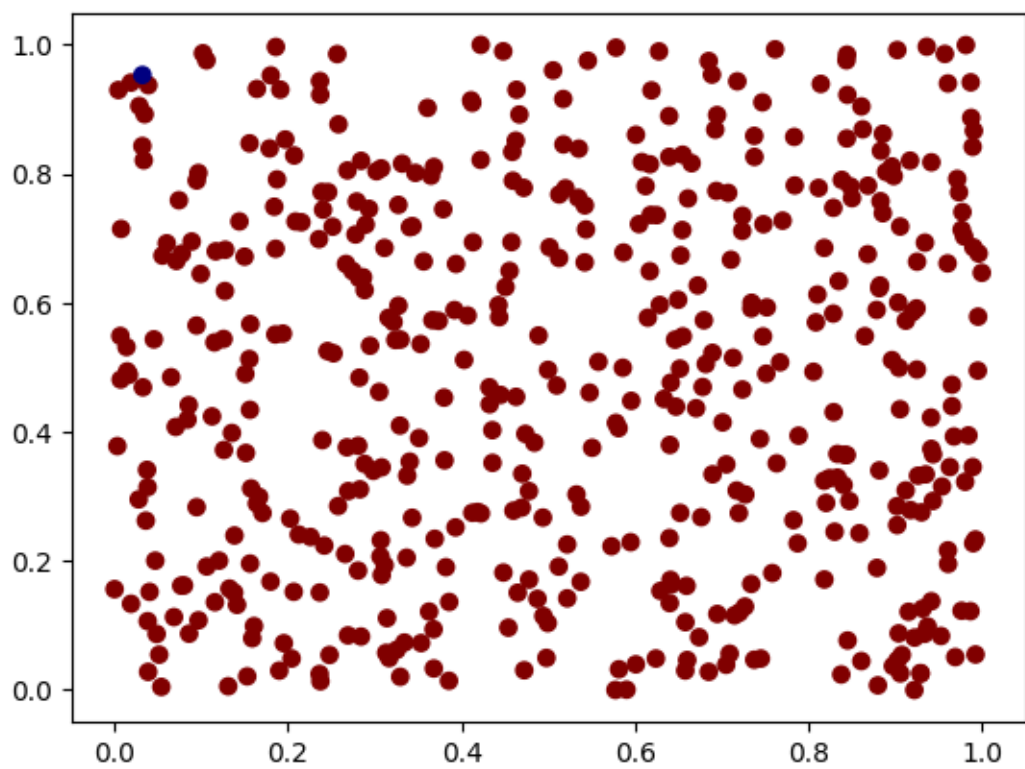
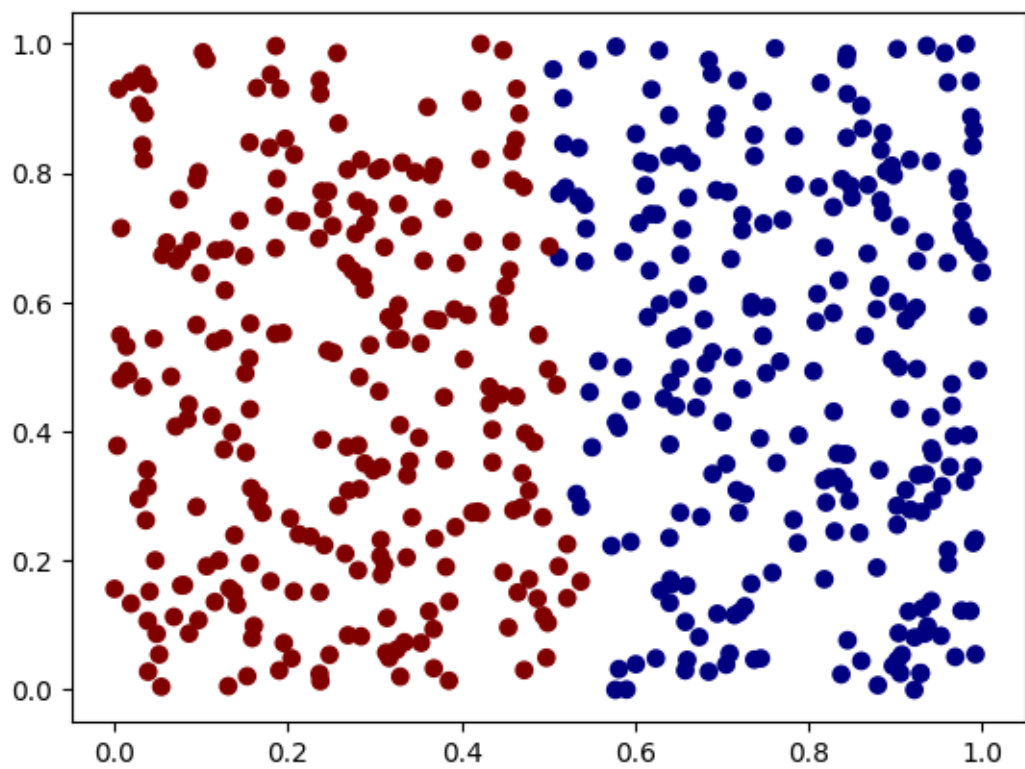
1.2

1.3 Nostructure

```
[11]: dataset = noStructure
x , y = dataset
x = MinMaxScaler().fit_transform(x)
plt.scatter(x[:, 0], x[:, 1], alpha=1.0, color='b')
# Clustering
# Within-Cluster Sum of Squares (WCSS) metric
wcss = []
model = []
for i in range(1, 11):
    model.append(KMeans(n_clusters=i).fit(x))
    wcss.append(model[i-1].inertia_)
wcscs = []
model = []
for i in range(1, 11):
    model.append(KMeans(n_clusters=i).fit(x))
    wcss.append(model[i-1].inertia_)
plt.plot(range(1, 11), wcss)
plt.title('Elbow Method')
plt.xlabel('Number of Clusters')
plt.ylabel('WCSS')
plt.show()
yp = model[1].fit_predict(x)
cmap = plt.cm.jet
fig , ax = plt.subplots()
ax.scatter(x[:, 0], x[:, 1], c=yp, cmap=cmap)
# DBSCAN
# Density-Based Spatial Clustering Applications With Noise
model = DBSCAN(eps=0.1, min_samples=10)
yp = model.fit_predict(x)
fig , ax = plt.subplots()
ax.scatter(x[:, 0], x[:, 1], c=yp + 1, cmap=cmap)
```



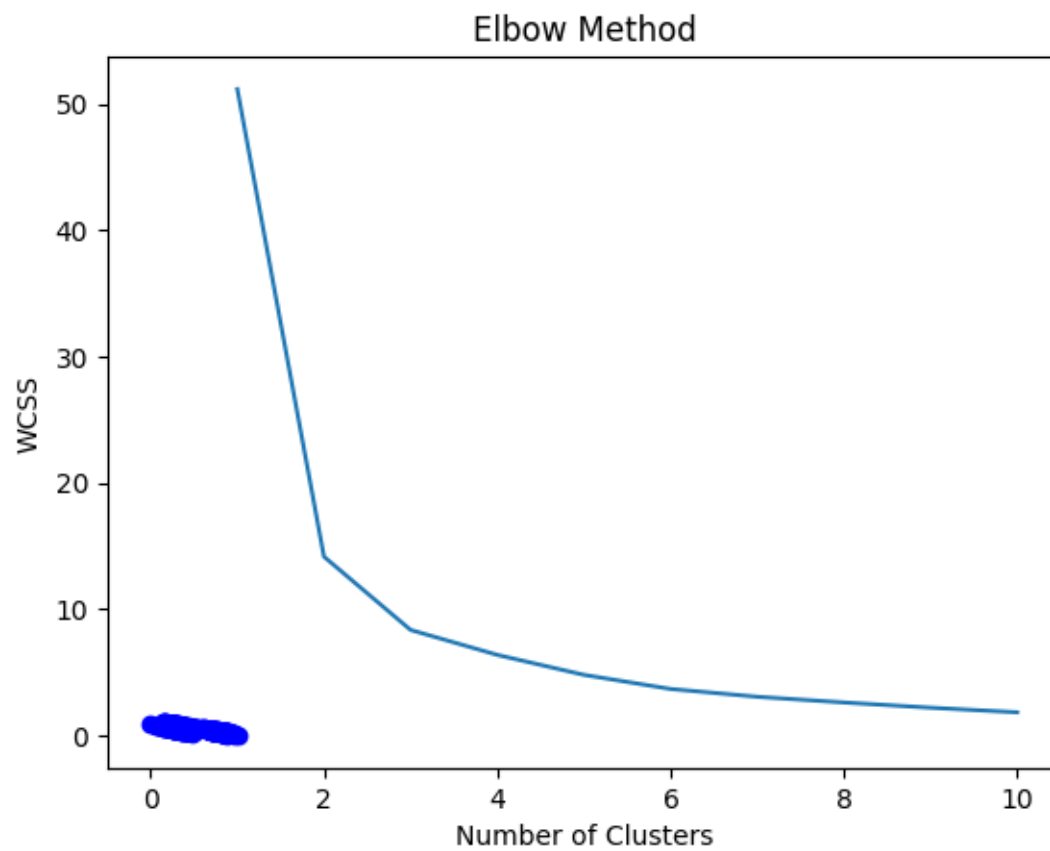
[11]: <matplotlib.collections.PathCollection at 0x1e7f76d4410>



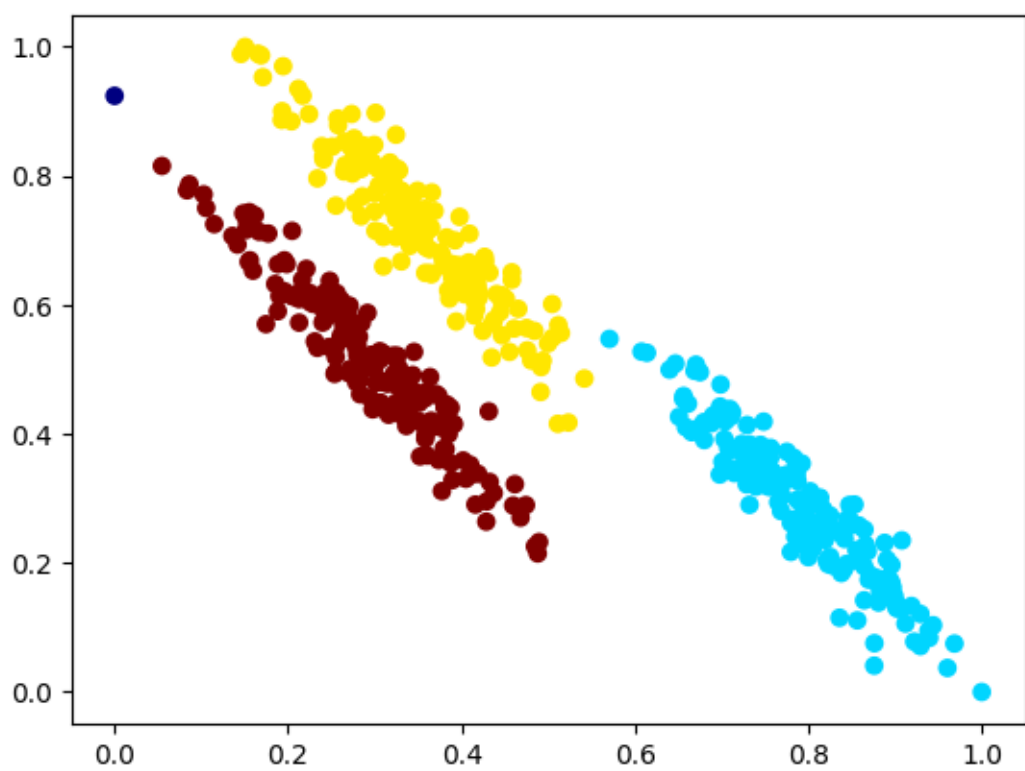
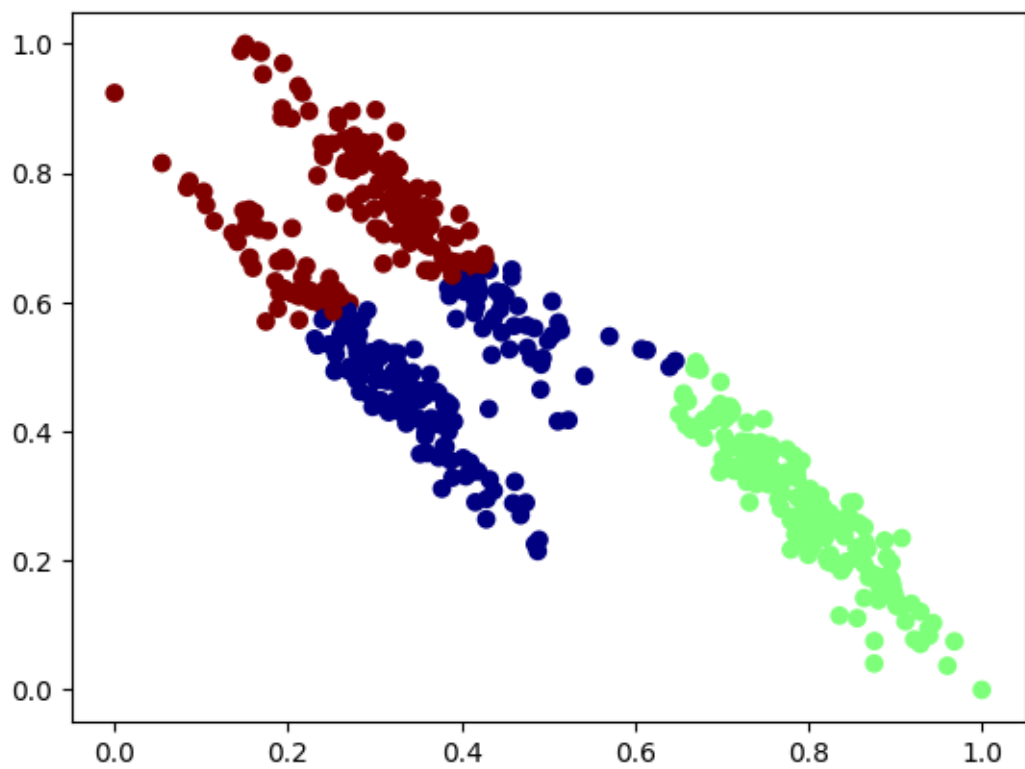
```

[12]: dataset = aniso
x , y = dataset
x = MinMaxScaler().fit_transform(x)
plt.scatter(x[:, 0], x[:, 1], alpha=1.0, color='b')
# Clustering
# Within-Cluster Sum of Squares (WCSS) metric
wcss = []
model = []
for i in range(1, 11):
    model.append(KMeans(n_clusters=i).fit(x))
    wcss.append(model[i-1].inertia_)
wcss = []
model = []
for i in range(1, 11):
    model.append(KMeans(n_clusters=i).fit(x))
    wcss.append(model[i-1].inertia_)
plt.plot(range(1, 11), wcss)
plt.title('Elbow Method')
plt.xlabel('Number of Clusters')
plt.ylabel('WCSS')
plt.show()
yp = model[2].fit_predict(x)
cmap = plt.cm.jet
fig , ax = plt.subplots()
ax.scatter(x[:, 0], x[:, 1], c=yp, cmap=cmap)
# DBSCAN
# Density-Based Spatial Clustering Applications With Noise
model = DBSCAN(eps=0.06, min_samples=5)
yp = model.fit_predict(x)
fig , ax = plt.subplots()
ax.scatter(x[:, 0], x[:, 1], c=yp + 1, cmap=cmap)

```

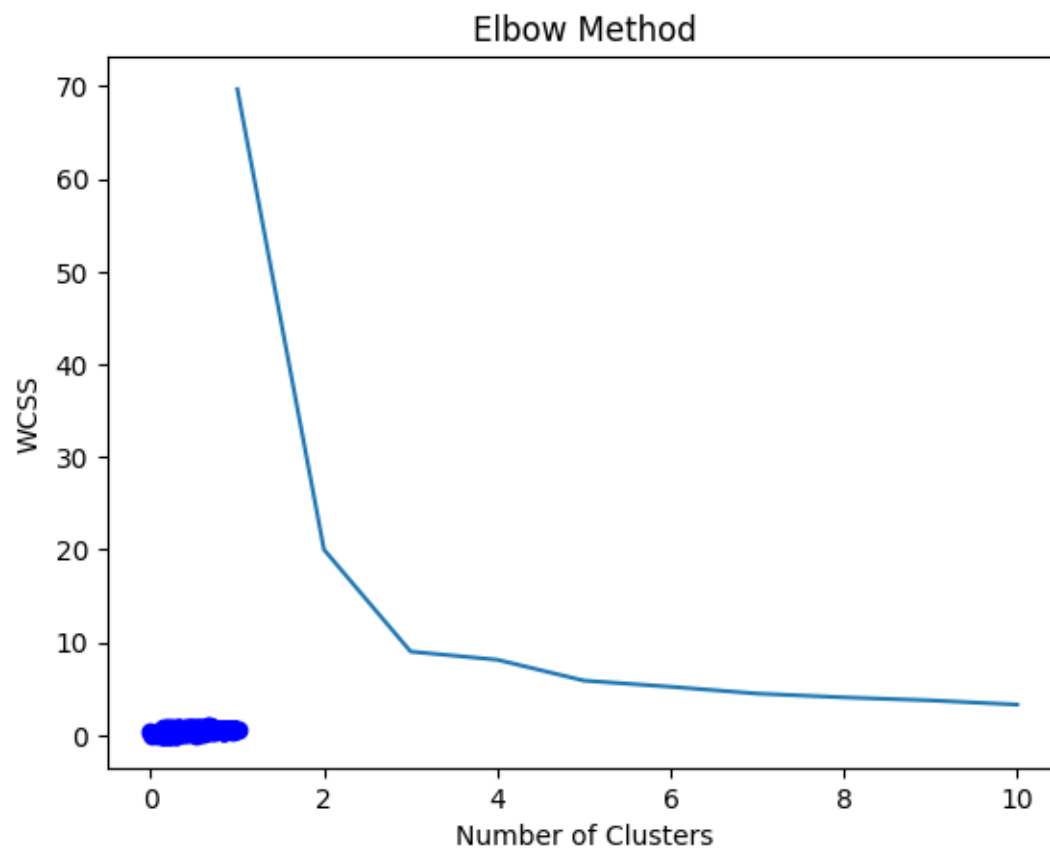


[12]: <matplotlib.collections.PathCollection at 0x1e7f629d310>

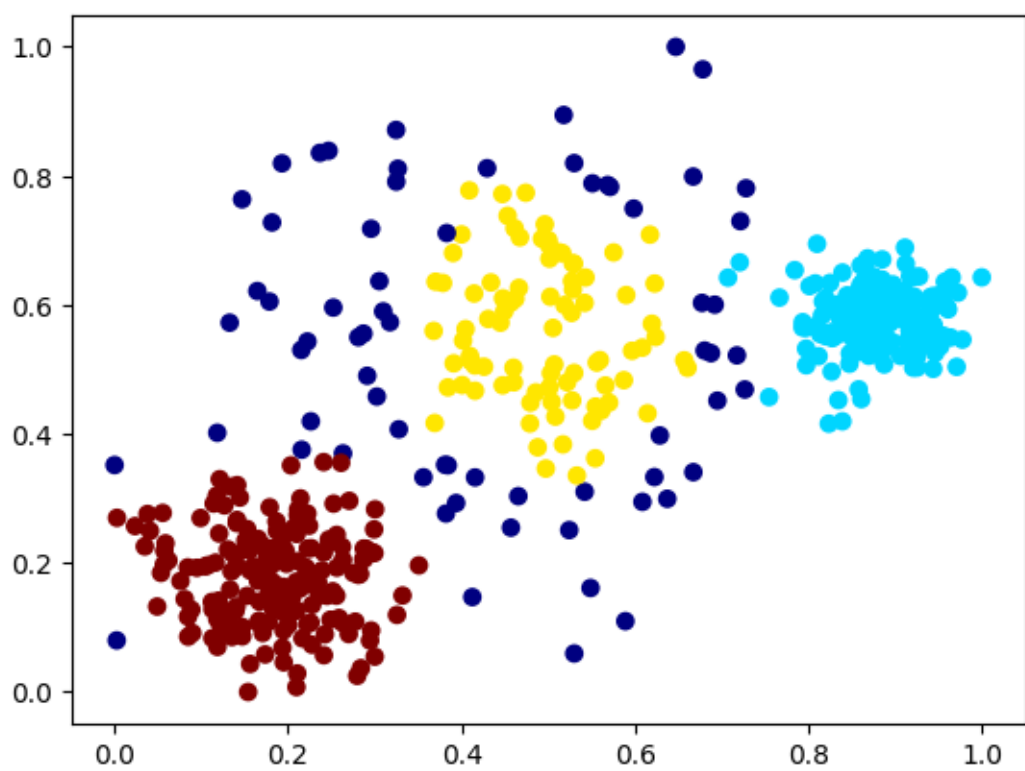
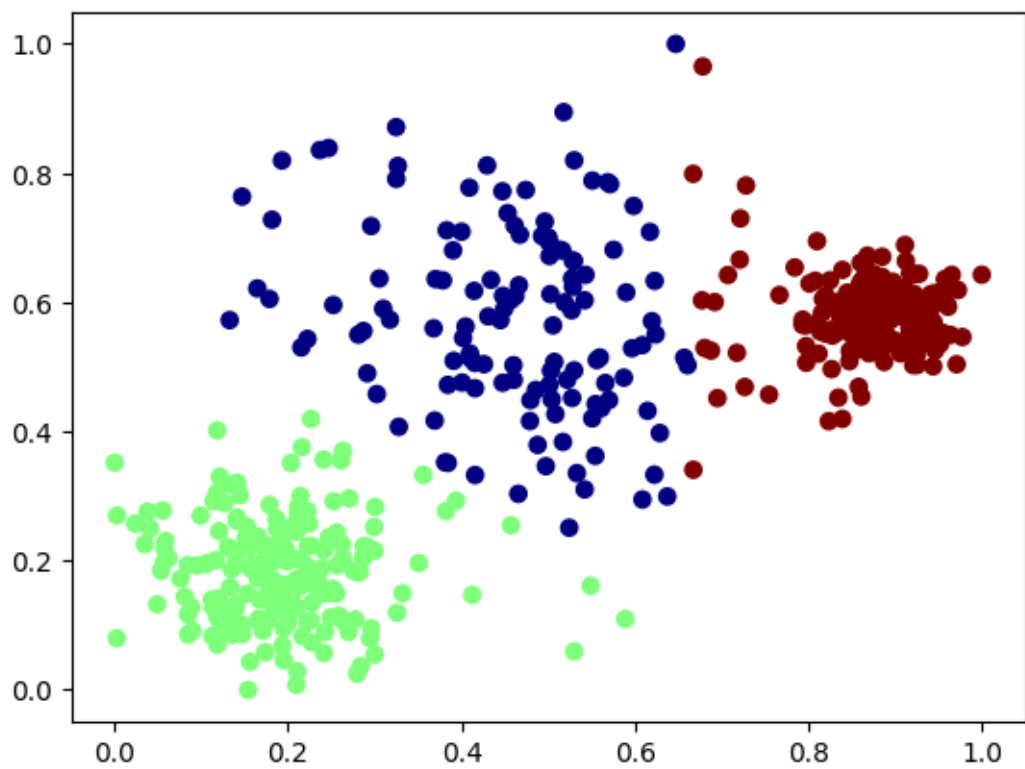


1.4 Varied

```
[13]: dataset = varied
x , y = dataset
x = MinMaxScaler().fit_transform(x)
plt.scatter(x[:, 0], x[:, 1], alpha=1.0, color='b')
# Clustering
# Within-Cluster Sum of Squares (WCSS) metric
wcss = []
model = []
for i in range(1, 11):
    model.append(KMeans(n_clusters=i).fit(x))
    wcss.append(model[i-1].inertia_)
wcss = []
model = []
for i in range(1, 11):
    model.append(KMeans(n_clusters=i).fit(x))
    wcss.append(model[i-1].inertia_)
plt.plot(range(1, 11), wcss)
plt.title('Elbow Method')
plt.xlabel('Number of Clusters')
plt.ylabel('WCSS')
plt.show()
yp = model[2].fit_predict(x)
cmap = plt.cm.jet
fig , ax = plt.subplots()
ax.scatter(x[:, 0], x[:, 1], c=yp, cmap=cmap)
# DBSCAN
# Density-Based Spatial Clustering Applications With Noise
model = DBSCAN(eps=0.07, min_samples=10)
yp = model.fit_predict(x)
fig , ax = plt.subplots()
ax.scatter(x[:, 0], x[:, 1], c=yp + 1, cmap=cmap)
```



[13]: <matplotlib.collections.PathCollection at 0x1e7f789aad0>



1.5 Conclusion

- 1.5.1 As demonstrated by the comparative results across all datasets, both the KMeans and DBSCAN algorithms are well-suited for clustering unlabeled data. A notable observation is that KMeans performs better on datasets with circular distributions, while DBSCAN is more effective for datasets with linear density patterns. The clustering tasks were carried out using the code developed during class sessions, applied consistently across various datasets.
- 1.5.2 An important insight emerged when working with the “noisy circle” dataset: DBSCAN yielded superior results. This is because, contrary to expectations, KMeans struggles to correctly cluster data with hollow circular structures, often misclassifying points due to its reliance on centroid-based partitioning.
- 1.5.3 In a separate file, an improved method for identifying more suitable clusters was implemented. This included defining reusable functions to streamline similar tasks and facilitate analysis. It is worth noting that this alternative approach is likely to produce even better clustering outcomes, especially when dealing with complex or irregular data distributions.